



## TALLER 2 - Desarrollo de una cuadrícula interactiva mediante coordenadas

Miguel Angel Diaz Hernandez

Taller de programación

Universidad Manuela Beltrán  
Ingeniería de Software  
Facultad de Ingenierías  
Bogotá, Colombia  
2026

## **Contexto**

Se pide al estudiante que represente una cuadrícula de círculos, todos transparentes al inicio. El usuario ingresa una coordenada (X, Y) y el programa identifica la posición correspondiente dentro de la cuadrícula, cambiando ese círculo de transparente a relleno. El ejercicio aplica conceptos de variables, coordenadas, estructuras repetitivas, condicionales, funciones y manejo de entradas por consola.

## **Objetivo**

Desarrollar un programa de consola que construya dinámicamente una cuadrícula de círculos de tamaño definido por el usuario (mínimo  $8 \times 8$ ), asigne una coordenada (X, Y) a cada posición, reciba una coordenada válida, cambie el estado del círculo correspondiente, registre las coordenadas seleccionadas y documente el proceso de desarrollo completo.

## Documentación del código

### ***Bloque 1: Declaración de la función para pedir cuadrícula***

```
def ingresar_dimensiones(pregunta1):
    while True:
        entrada = input(pregunta1)
        if entrada.isdigit() and int(entrada) >= 8:
            return int(entrada)
        else:
            print("Ingrese un numero entero mayor o igual a 8")
```

Este bloque declara la función “ingresar\_dimensiones” que recibe las dimensiones de la cuadrícula de tamaño variable con un mínimo de  $8 \times 8$ . Se apoya en un ciclo while True que no permite avanzar hasta recibir un valor correcto: primero confirma que el texto ingresado contenga solo dígitos, evitando que el programa se detenga si el usuario escribe letras, y luego confirma que el número cumpla el mínimo exigido. La misma función se reutiliza dos veces, una para el ancho y otra para el alto, evitando duplicar lógica.

### ***Bloque 2: Construcción de la Cuadrícula***

```
def iniciar_cuadrícula(ancho, alto):
    return [['○' for _ in range(ancho)] for _ in range(alto)]
```

En este bloque se construye la matriz (lista de listas), donde cada fila representa un valor de Y y cada columna un valor de X. El tamaño, se decide con los valores capturados en el bloque anterior. Cada celda inicia con el símbolo [○], que representa un círculo vacío dentro de un cuadrado, cumpliendo el requisito de que la cuadrícula inicie completamente transparente.

### ***Bloque 3: Validación y Cambio de Estado***

```
def validar(x, y, ancho, alto):
    return 1 <= x <= ancho and 1 <= y <= alto

def rellenar(cuadrícula, x, y):
    fila = y - 1
    columna = x - 1
    cuadrícula[fila][columna] = '●'
```

Estas dos funciones forman una sola unidad lógica: primero se valida y después se actúa. La función validar comprueba que la coordenada existe dentro de los límites actuales de la cuadrícula. Solo si la respuesta es verdadera se invoca rellenar, que traduce la coordenada ingresada por el usuario al índice real de la lista y cambia el símbolo de esa posición a [●]. Separar la decisión de la acción evita que el programa modifique la cuadrícula con datos fuera de rango.

### ***Bloque 4: Visualización en Consola***

```
def imprimir_cuadrícula(cuadrícula):
    for fila in cuadrícula:
        print(' '.join(fila))
```

Este bloque recorre la matriz fila por fila y convierte cada fila en una sola línea de texto separada por espacios, mediante ' '.join(fila) en lugar de un segundo ciclo anidado. Es la única función encargada de mostrar el estado visual de la cuadrícula y se reutiliza tanto al iniciar el programa como cada vez que el usuario solicita verla o después de reiniciarla.

## ***Bloque 5: Flujo Principal y Menú de Opciones***

```

while True:
    print("1. Seleccionar coordenada")
    print("2. Ver cuadrícula")
    print("3. Ver seleccionadas")
    print("4. Reiniciar selecciones")
    print("5. Salir")
    entrada_opcion = opcion = (input("Opcion: "))
    if entrada_opcion.isdigit():
        opcion = int(entrada_opcion)
    else:
        print("Opcion invalida")
        continue
    if opcion == 1:
        entrada_x = input("Ingreso X: ")
        entrada_y = input("Ingreso Y: ")
        if entrada_x.isdigit() and entrada_y.isdigit():
            x = int(entrada_x)
            y = int(entrada_y)
            if validar(x, y, ancho, alto):
                rellenar(cuadrícula, x, y)
                seleccionadas.append(f"{x}, {y}")
            else:
                print("Coordenada inexistente")
        else:
            print("Se deben ingresar numeros enteros")

    elif opcion == 2:
        imprimir_cuadrícula(cuadrícula)
    elif opcion == 3:
        print(seleccionadas)
    elif opcion == 4:
        cuadrícula = iniciar_cuadrícula(ancho,alto)
        seleccionadas = []
    elif opcion == 5:
        break
    else:
        print("Opcion invalida")

```

Este bloque mantiene la aplicación activa mientras el usuario no elija salir y conecta todos los bloques anteriores según la opción elegida. La opción del menú se valida igual que las coordenadas, confirmando que sea un número antes de compararla, y usa continue para volver a mostrar el menú sin intentar procesar una opción inválida.

**Bloque 6: Registro y Consulta de Coordenadas Seleccionadas**

```
seleccionadas = []  
...  
seleccionadas.append(f"{x}, {y}")  
...  
elif opcion == 3:  
    print(seleccionadas)  
elif opcion == 4:  
    cuadrícula = iniciar_cuadrícula(ancho, alto)  
    seleccionadas = []
```

Este bloque cumple el requisito de mantener un historial de coordenadas seleccionadas. Cada vez que se rellena un círculo, su coordenada se agrega a la lista “seleccionadas”. La opción 3 del menú imprime esa lista, y la opción 4, correspondiente a reiniciar, no solo reconstruye la cuadrícula desde cero, sino que también vacía el historial, dejando el programa en el mismo estado que al iniciar.

## Conclusiones

- Organizar el programa en bloques con una única responsabilidad facilitó identificar y corregir errores de forma independiente durante el desarrollo.
- Separar la validación de la coordenada del cambio de estado del círculo evitó que coordenadas fuera de rango modificaran la cuadrícula.
- Validar cada entrada de teclado antes de convertirla a número entero evitó que el programa se detuviera al recibir datos inesperados, tanto en las coordenadas como en las opciones del menú.
- El programa se ejecuta completamente en consola, sin interfaces gráficas ni componentes web, cumpliendo el requisito indicado por el docente.
- El proyecto cumple con la totalidad de los requerimientos funcionales (RF01 a RF10) planteados en la guía de la actividad.